

Prova - Programação Estruturada

a) remover Repetidos

```
int removerRepetidos(int v[], int tam) {  
    int i, novoTam;
```

```
    if (tam <= 0) return 0;
```

```
    novoTam = 1;
```

```
    for (i = 1; i < tam; i++) {
```

```
        if (v[i] != v[novoTam - 1]) {
```

```
            v[novoTam] = v[i];
```

```
            novoTam++;
```

```
        }
```

```
    }
```

```
    return novoTam;
```

```
}
```

b) ordenar - bubble sort

```
void ordenar(int v[], int tam) {
```

```
    int i, j, aux;
```

```
    for (i = 0; i < tam - 1; i++) {
```

```
        for (j = 0; j < tam - 1 - i; j++) {
```

```
            if (v[j] > v[j + 1]) {
```

```
                aux = v[j];
```

```
                v[j] = v[j + 1];
```

```
v[j + 1] = aux;
```

```
                v[j + 1] = aux;
```

```
            }
```

```
        }
```

```
    }
```

2,0/2,0

2,0/2,0

a) ~~procurar~~ procurar Primos
 int primoUsado Vektor (int numero, int v[], int qtdPrimos) {
 int i;

if (numero < 2) return 0;

for (i = 0; i < qtdPrimos; i++) {
 if (numero % v[i] == 0) {
 return 0;
 }
}

return 1;

}
 void procurarPrimos (int v[], int tam) {
 int qtd = 0;
 int candidato = 2;

while (qtd < tam) {
 if (primoUsado Vektor (candidato, v, qtd)) {
 v[qtd] = candidato;
 qtd++;
 }
 candidato++;

}

d) maiorParbuka

```
void maiorParbuka (int mat[1000][1000], int dim,  
                  int col, int v[1]) {  
    int i, j, maior;
```

```
    for (i = 0; i < dim; i++) {
```

```
        maior = mat[i][0];
```

```
        for (j = 1; j < col; j++) {
```

```
            if (mat[i][j] > maior)
```

```
                maior = mat[i][j];
```

```
        }
```

```
        v[i] = maior;
```

```
    }
```

```
}
```

2,0/2,0

e) invertePalavras

```
void inverteTraco (char str[1], int mi, int fim) {
```

```
    char aux;
```

```
    while (mi < fim) {
```

```
        aux = str[mi];
```

```
        str[mi] = str[fim];
```

```
        str[fim] = aux;
```

```
        mi++; fim--;
```

```
    }
```

```
}
```

```
void invertePalavras (char str[]) {
```

```
    int i = 0, menor;
```

```
    while (str[i] != '\0') {
```

```
        if (str[i] == '"') i++;
```

```
        else {
```

2,0/2,0

```

micro = i;
while (str[i] != '\0' && str[i] != '\n') i++;
return str (str, micro, i-1);

```

```

}

```

```

}

```

```

}

```

```

{ (i++); while (i < n) {
    ; str[i] = str[i-1];
}

```

```

{ (i++); while (i < n) {
    ; str[i] = str[i-1];
}

```

```

; str[i] = str[i-1];

```

```

; str[i] = str[i-1];

```

```

; str[i] = str[i-1];

```

```

{ (i++); while (i < n) {
    ; str[i] = str[i-1];
}

```

```

{ (i++); while (i < n) {
    ; str[i] = str[i-1];
}

```

```

; str[i] = str[i-1];

```

```

; str[i] = str[i-1];

```

```

; str[i] = str[i-1];

```

```

; str[i] = str[i-1];

```

```

{ (i++); while (i < n) {
    ; str[i] = str[i-1];
}

```